

Documentación Técnica

Sistema de Detección y Alerta de Vencimiento de Documentos

Este documento describe, de forma integral, la solución implementada sobre el NAS Synology de Recicley para automatizar la detección de permisos, licencias y documentos afines próximos a vencer, así como el envío de alertas por correo electrónico a los responsables correspondientes. Incluye además el seguimiento independiente de los acuses de recibo relacionados con dichos trámites.

Campo	Detalle
Responsable	Leonardo Aguilar Martínez — Depto. TI
Servidor	Recicley_Server (Synology NAS)
Acceso remoto	recicleymx.us2.quickconnect.to
Script	<code>script.py</code> + <code>carpetas.txt</code> (misma carpeta), ejecutado vía wrapper Bash desde el Programador de tareas
Fecha de elaboración	27 de julio de 2026
Última actualización	10 de agosto de 2026 — sincronizada con la versión de <code>script.py</code> en uso
Estado	Funcional — en producción, con la lista completa de carpetas de Calidad-Gestoría (73 rutas)

1. Objetivo

Automatizar el monitoreo del ciclo de vida documental de los permisos y licencias operativas de Recicley, eliminando el seguimiento manual mediante hojas de cálculo o recordatorios informales. El sistema revisa periódicamente el repositorio de documentos, calcula el tiempo restante antes de cada vencimiento y notifica proactivamente a los equipos responsables, garantizando que ningún permiso expire sin haber sido gestionado a tiempo. Adicionalmente, da seguimiento por separado a los acuses de recibo relacionados (documentos cuyo nombre inicia con `ACUSE_`), reportándolos en un segundo correo independiente del de vencimientos.

2. Arquitectura General

La solución combina el almacenamiento nativo del NAS, un script de automatización en Python y el servicio de correo de Google Workspace, orquestados desde las herramientas de administración de DSM (el sistema operativo del Synology).

Componente	Función dentro del flujo
Carpetas de documentos	Repositorio de PDFs dentro de la jerarquía de Calidad-Gestoría, organizado por tipo, con la fecha de vencimiento incluida en el nombre del archivo
Programador de tareas (DSM)	Ejecuta el script de verificación de forma automática y periódica
Script de Python (<code>script.py</code>)	Escanea las carpetas de la lista blanca, calcula vencimientos, separa acuses y construye los reportes
SMTP de Gmail Workspace	Canal de envío autenticado, configurado a nivel de sistema en DSM
Variable de entorno protegida	Resguarda la contraseña de la cuenta de correo fuera del código fuente
Google Drive (Cloud Sync)	Repositorio espejo de los documentos, como respaldo en la nube

Flujo resumido del proceso

1. Los documentos se almacenan en el NAS, dentro del árbol de carpetas de Calidad-Gestoría, con una convención de nombres estandarizada.
2. Una tarea programada ejecuta `script.py` automáticamente en un horario definido.
3. El script recorre la lista blanca de carpetas y clasifica cada documento próximo a vencer o vencido en dos grupos: **vencimientos** (permisos y licencias) y **acuses** (nombre con prefijo `ACUSE_`).
4. Si existen coincidencias, construye y envía un correo consolidado y agrupado por carpeta para cada grupo, de forma independiente.
5. En paralelo, Cloud Sync mantiene una copia de respaldo de los documentos en Google Drive.

3. Configuración del Entorno en el NAS

3.1 Acceso administrativo

La administración del NAS se realiza a través del panel web de DSM, accesible de forma remota mediante QuickConnect, el servicio de Synology que permite conectarse al servidor sin depender de la IP local de la red donde está físicamente instalado.

GeneralRedAlmacenamientoServicioAnálisis del dispositivo

^ Información básica

Nombre del servidor	Recicley_Server
DNS	192.168.101.1
Pasarela predeterminada	192.168.101.1
Grupo de trabajo	WORKGROUP
Servidor WINS	<No configurado>
DDNS	recicleyservr.synology.me (Normal)

^ LAN 1

Estado de la conexión	Conectado
Dirección MAC	90-09-D0-75-EE-ED
Dirección IP	192.168.101.103 (DHCP)
Máscara de subred	255.255.255.0
Estado de red	1000 Mbps, Dúplex completo, MTU 1500

^ LAN 2

Estado de la conexión	Conectado
Dirección MAC	90-09-D0-75-EE-EE
Dirección IP	192.168.101.102 (DHCP)
Máscara de subred	255.255.255.0
Estado de red	1000 Mbps, Dúplex completo, MTU 1500

Panel de Red de DSM — nombre del servidor, DNS y configuración DDNS utilizada para el acceso remoto.

3.2 Estructura y lista blanca de carpetas monitoreadas

El script no recorre todo el NAS: opera sobre una **lista blanca** (la variable `CARPETAS` , cargada en tiempo de ejecución desde `carpetas.txt` — ver 4.5) de **73 rutas** específicas, todas dentro del volumen 4, bajo el árbol documental de Calidad-Gestoría. Cualquier carpeta que no esté explícitamente incluida en esta lista no es monitoreada. La ruta base común es:

```
/volume4/Contraloría/1.Calidad-Gestoría/
```

Categoría	Subcarpetas monitoreadas
1. Permisos de Recolección y Traslado — Recicley MX	Estado de México, Queretaro, RAMIR RMX, Residuos Peligrosos (4)
1. Permisos de Recolección y Traslado — RMO	Coahuila, Estado de México, Guadalajara, Guanajuato, Herreros, Monterrey, Nave 27, Puebla, Ramir RMO, Residuos Peligrosos (10)
2. Permisos Planta de Separación (Segregación) — Recicley MX	Carpeta completa, sin subdivisión adicional (1)
3. Licencias de Funcionamiento — Foráneos	Guadalajara, Puebla, San Luis Potosí, Xhala (4)
3. Licencias de Funcionamiento — Toluca	14 subcarpetas numeradas: Báscula, CAF-Contables, Composta Viejita, Destrucciones, Ducto de gas, El Dorado, Madera Hidalgo, Madera Independencia, Segregación I, Madera Matamoros, Segregación II, Plásticos, Triángulo Chatarra, Segregación III (14)
4. Permisos de Aprovechamiento — RMO	Estado de México (Báscula, CAF-Contables, Destrucciones, Plásticos, Composta), Guadalajara, Monterrey (7)
18. Cedula de Zonificación	18 subcarpetas numeradas: Xhala 2, Composta viejita, Composta 2, Patio de maniobras, Reci 16-Taller, Segregación III, Frente a Johnson, Madera Matamoros, Ducto de gas, El Dorado, Almacén Johnson, Reci 012, Chatarra, Plásticos, Cruz Blanca, Triángulo Chatarra, Segregación I y II, Xhala 1 (18)
19. Movilidad Vial GDL y SLP	Carpeta completa, sin subdivisión adicional (1)
20. Factibilidad Vial	San Luis Potosí, Guadalajara (2)
21. NISS — Recibo Pago de agua y Saneamiento	Báscula, Reci 16-Taller, CAF-Contables, Destrucciones, Estacionamiento, Johnson, Plásticos (7)
22. Poliza vs Daños Vigente al 30052027	Carpeta completa, sin subdivisión adicional (1)
23. Factibilidad de agua y drenaje	Reci 16-Taller, CAF-Contables, Composta Viejita (3)
24. Autorización para depósito de RSU en sitio SLP	Carpeta completa, sin subdivisión adicional (1)

Recorrido recursivo dentro de cada ruta: el script revisa subcarpetas dentro de cada una de las 73 rutas listadas (por ejemplo, sub-expedientes creados dentro de "Coahuila"), pero no descubre carpetas nuevas fuera de la lista. Para monitorear una carpeta adicional basta con agregar una línea a `carpetas.txt` — no requiere editar ni volver a desplegar `script.py` (ver 4.5).

3.3 Convención de nomenclatura

Para que el sistema pueda calcular automáticamente el vencimiento, cada archivo debe seguir un formato de nombre estandarizado, con la fecha de expiración incluida al final:

```
DESCRIPCION_DOCUMENTO_DD-MM-AAAA.pdf
```

Ejemplos aplicados:

- LICENCIA_AMBIENTAL_30-11-2026.pdf
- PERMISO_TRANSPORTE_RESIDUOS_15-08-2025.pdf
- POLIZA_SEGURO_20-01-2026.pdf

Documentos de acuse siguen la misma convención, con el prefijo adicional ACUSE_ :

- ACUSE_PERMISO_TRANSPORTE_15-08-2025.pdf

⚠ Cambio de comportamiento respecto a versiones previas: el patrón vigente (`PATRON = re.compile(r"(\d{2}-\d{2}-\d{4})\.pdf$")`) exige que la fecha esté seguida *exactamente* de .pdf al final del nombre, sin tolerar doble extensión (.pdf.pdf). Una versión anterior del script sí toleraba esa doble extensión. Como se observa en la captura siguiente, 3 de los 4 archivos de prueba en File Station tienen doble extensión .pdf.pdf y **no serían detectados** por el script actual. Se recomienda renombrar los archivos afectados o restaurar la tolerancia en la expresión regular si el caso sigue ocurriendo en producción.

<

>

🔄

Destrucciones > PRUEBAS CODIGO

★

Q- Bus

Crear ▾

Cargar ▾

Acción ▾

Herramientas ▾

Configuración

Nombre ▾	Tamaño	Tipo de archivo	Fecha de modificación
 PERMISO_PRUEBA_28-07-2026.pdf	52.4 KB	PDF Archivo	27/07/2026 08:04:58
 PERMISO_PRUEBA2_28-07-2026.pdf.pdf	208.5 KB	PDF Archivo	24/07/2026 17:03:02
 PERMISO_PRUEBA3_28-07-2026.pdf.pdf	193.6 KB	PDF Archivo	23/07/2026 08:24:33
 PERMISO_PRUEBA4_28-07-2026.pdf.pdf	14.5 MB	PDF Archivo	25/05/2026 16:42:17
 script.py	2.7 KB	PY Archivo	27/07/2026 11:38:42

Carpeta de pruebas en File Station — nótese la doble extensión .pdf.pdf en 3 de los 4 archivos, un caso no cubierto por el patrón vigente. El archivo script.py visible aquí es el componente central de la solución.

3.4 Notificaciones por correo electrónico (SMTP)

DSM permite centralizar el envío de correo del sistema a través de un servidor SMTP externo. Se configuró la cuenta institucional de Google Workspace del departamento de TI como remitente autorizado, en: *Panel de control → Notificación → Correo electrónico → Configurar remitente.*

Configurar el remitente

Las notificaciones del sistema se enviarán desde la dirección de correo electrónico de este remitente.

Proveedor de servicios:

Servidor SMTP personalizado

Servidor SMTP:

smtp.gmail.com

Puerto SMTP:

465

☒ Se requiere autenticación

Nombre de usuario:

ti@recicleymx.com

Contraseña:

.....

☒ Es necesaria una conexión segura (SSL/TLS)

Nombre del remitente:

Correo electrónico del remitente:

Cancelar

Guardar

Configuración del remitente SMTP: servidor, puerto, autenticación y conexión segura.

Parámetro	Valor
Servidor SMTP	smtp.gmail.com
Puerto	465 (SSL)
Autenticación	Habilitada, con contraseña de aplicación de Google
Conexión segura	SSL/TLS habilitada
Cuenta remitente	ti@recicleymx.com

Adicionalmente, se definieron los perfiles de destinatarios que reciben las notificaciones del propio sistema DSM, útiles para monitorear el estado general del servidor:

Correo electrónico
Servicio Push
Webhooks
Reglas
Eventos

Cuenta de Synology

☒ Reciba notificaciones directamente en su Cuenta Synology cuando se produzcan cambios o errores en el estado del sist servidor de correo electrónico de Synology.

Cuenta de Synology:
ti@recicleymx.com

Regla:
All

Enviar correo electrónico de prueba

Correo electrónico

Reciba notificaciones a través de correo electrónico cuando se produzcan cambios o errores en el estado del sistema.

Remitente

Correo electrónico del remitente:
ti@recicleymx.com (Recicley - Alerta por vencimiento)

Configurar

Perfiles de destinatarios

Agregar
Editar
Eliminar
Enviar correo electrónico de prueba

Correo electrónico
asistencia.ti@recicleymx.com

Perfiles de destinatarios configurados para las notificaciones del sistema.

3.5 Automatización con el Programador de tareas

La ejecución periódica del script se gestiona mediante el Programador de tareas de DSM, sin depender de una conexión activa por parte del usuario. La tarea se ejecuta de forma silenciosa en segundo plano, según el horario configurado.

Parámetro	Valor configurado
Nombre de la tarea	Verificación de vencimientos
Usuario de ejecución	root
Programación	Diaria, 08:00 h (ajustable)
Comando ejecutado	bash /volume1/scripts/ejecutar.sh
Notificación de resultado	Envío del log de ejecución por correo

Para mantener la contraseña de la cuenta de correo fuera del script principal, la tarea invoca un pequeño script intermediario que define la credencial como variable de entorno antes de ejecutar Python:

```
#!/bin/bash
export GMAIL_APP_PASS="xxxxxxxxxxxxxxxxx"
python3 "/volume1/scripts/script.py"
```



Recicley - Alerta por vencimiento <ti@recicleymx.com>
para mí ▼

El programador de tareas ha completado una tarea programada.

Tarea: Permisos

Hora de inicio: Mon, 27 Jul 2026 12:39:08 -0500

Hora de finalización: Mon, 27 Jul 2026 12:39:10 -0500

Estado actual: 0 (Normal)

Salida estándar/error:

Correo enviado correctamente.

Desde Recicley_Server

Confirmación de ejecución exitosa: la tarea corrió y el correo se envió correctamente.

4. Script de Automatización en Python

4.1 Propósito

El script `script.py` es el componente central de la solución. Recorre las 73 carpetas de la lista blanca, interpreta la fecha de vencimiento embebida en el nombre de cada archivo PDF, y determina si algún documento requiere atención según el umbral de días definido. Separa los resultados en dos grupos — vencimientos y acuses (prefijo `ACUSE_`) — y arma y envía, cuando corresponde, un correo por cada grupo, con el detalle agrupado por carpeta y ordenado por fecha de vencimiento.

4.2 Variables de configuración

Variable	Descripción
CARPETAS	Lista blanca de 73 rutas a escanear, de forma recursiva (incluye subcarpetas); se carga en tiempo de ejecución desde <code>carpetas.txt</code>
_ARCHIVO_CARPETAS	Nombre del archivo de texto que contiene la lista blanca (<code>carpetas.txt</code>), leído desde la misma carpeta que <code>script.py</code>
DIAS_ALERTA	Umbral de días para activar la alerta (actualmente 30)
GMAIL_USER	Cuenta institucional remitente (<code>ti@recicleymx.com</code>)
DESTINATARIO_PRINCIPAL	Correo principal que recibe la alerta de vencimientos
CC	Lista de correos adicionales en copia para la alerta de vencimientos
DESTINATARIO_ACUSES	Correo principal que recibe el listado de acuses en seguimiento
CC_ACUSES	Lista de correos adicionales en copia para el correo de acuses
PREFIJO_ACUSE	Prefijo de nombre de archivo que identifica un accuse (<code>ACUSE_</code>), reportado por separado
GMAIL_APP_PASS	Credencial leída del entorno del sistema, no expuesta en el código

4.3 Lógica de funcionamiento

1. Al importarse, carga `CARPETAS` leyendo `carpetas.txt` (misma carpeta que el script) y componiendo cada línea con `_BASE_CALIDAD`.
2. Recorre cada una de las 73 rutas resultantes, incluyendo todas sus subcarpetas; si una ruta no existe en el NAS, lo reporta en consola y continúa sin detener la ejecución.
3. Aplica una expresión regular para extraer la fecha de vencimiento del nombre de cada archivo PDF.
4. Calcula los días restantes entre hoy y la fecha de vencimiento de cada documento; descarta los que están fuera del umbral definido por `DIAS_ALERTA`.
5. Clasifica cada documento dentro del umbral como "por vencer" (días restantes) o "VENCIDO" (fecha ya superada).
6. Separa los documentos en dos listas según el nombre: los que inician con `ACUSE_` van a la lista de **acuses**; el resto, a la lista de **vencimientos**.
7. Agrupa cada lista por carpeta contenedora y ordena cada grupo por fecha de vencimiento (más urgente primero).
8. Construye un cuerpo de correo en texto plano por cada lista no vacía, con encabezado, fecha de generación, total de documentos y el detalle agrupado por carpeta.

9. Envía el correo de vencimientos (si aplica) al destinatario principal, con copia a los responsables definidos en `CC`.
10. Envía el correo de acuses (si aplica) de forma independiente, al destinatario y copias definidos para acuses.

4.4 Código completo

El listado de las 73 rutas ya no vive en el código; se lee de `carpetas.txt` (ver 3.2 y 4.5).

```
import os
import re
import smtplib
from datetime import date, datetime
from email.mime.text import MIMEText
from collections import defaultdict

# ===== CONFIGURACIÓN =====

_BASE_CALIDAD = "/volume4/Contraloría/1.Calidad-Gestoría"

_ARCHIVO_CARPETAS = "carpetas.txt"

def _cargar_carpetas(nombre_archivo=_ARCHIVO_CARPETAS):
    """Lee la lista blanca de carpetas desde un archivo de texto plano junto
    a este script: una ruta relativa a _BASE_CALIDAD por línea. Ignora
    líneas vacías y comentarios (que inician con #)."""
    ruta = os.path.join(os.path.dirname(os.path.abspath(__file__)), nombre_archivo)
    with open(ruta, encoding="utf-8") as f:
        return [
            f"{_BASE_CALIDAD}/{linea.strip()}"
            for linea in f
            if linea.strip() and not linea.strip().startswith("#")
        ]

CARPETAS = _cargar_carpetas()

DIAS_ALERTA = 30

GMAIL_USER = "ti@recicleymx.com"
DESTINATARIO_PRINCIPAL = "legal.permisos@recicleymx.com"
CC = ["leonardogr118@gmail.com"]

DESTINATARIO_ACUSES = "legal.permisos@recicleymx.com"
CC_ACUSES = ["leonardogr118@gmail.com"]

GMAIL_APP_PASS = os.environ.get("GMAIL_APP_PASS")
PREFIJO_ACUSE = "ACUSE_"
# =====

PATRON = re.compile(r"(\d{2}-\d{2}-\d{4})\.pdf$", re.IGNORECASE)
```

```

def escanear():
    """Recorre las carpetas y separa los documentos en dos listas:
    vencimientos (permisos normales) y acuses (nombre inicia con ACUSE_)."""
    vencimientos = []
    acuses = []

    for carpeta in CARPETAS:
        if not os.path.isdir(carpeta):
            print(f"Aviso: la carpeta no existe -> {carpeta}")
            continue
        for raiz, _, archivos in os.walk(carpeta):
            for nombre in archivos:
                m = PATRON.search(nombre)
                if not m:
                    continue
                try:
                    vence = datetime.strptime(m.group(1), "%d-%m-%Y").date()
                except ValueError:
                    continue

                restantes = (vence - date.today()).days
                if restantes > DIAS_ALERTA:
                    continue

                estado = "VENCIDO" if restantes < 0 else f"{restantes} dias restantes"
                registro = (nombre, vence, estado, raiz)

                if nombre.upper().startswith(PREFIJO_ACUSE):
                    acuses.append(registro)
                else:
                    vencimientos.append(registro)

    return vencimientos, acuses

def _agrupar_por_carpeta(documentos):
    """Agrupar una lista de documentos por su carpeta contenedora,
    ordenando cada grupo por fecha de vencimiento (mas urgente primero)."""
    grupos = defaultdict(list)
    for nombre, vence, estado, carpeta in documentos:
        grupos[carpeta].append((nombre, vence, estado))
    for carpeta in grupos:
        grupos[carpeta].sort(key=lambda x: x[1])
    return dict(sorted(grupos.items()))

def _construir_cuerpo(documentos, encabezado):
    """Genera un cuerpo de correo en texto plano, legible y agrupado por carpeta."""
    total = len(documentos)
    grupos = _agrupar_por_carpeta(documentos)

    ANCHO = 70
    lineas = [encabezado, f"Generado: {datetime.now().strftime('%d-%m-%Y %H:%M')}", "=" * ANCHO,
              f"Total de documentos: {total}", ""]

    for carpeta, items in grupos.items():

```

```

        lineas.append("-" * ANCHO)
        lineas.append(f"CARPETA: {carpeta}")
        lineas.append("-" * ANCHO)
        for nombre, vence, estado in items:
            lineas.append(f"    • {nombre}")
            lineas.append(f"        Vence: {vence.strftime('%d-%m-%Y')} | Estado: {estado}")
        lineas.append("")

    lineas.append("=" * ANCHO)
    lineas.append("Correo generado automaticamente por el sistema de alertas de Recicley.")
    return "\n".join(lineas)

def _enviar(destinatario_principal, cc, asunto, cuerpo):
    msg = MIMEText(cuerpo, "plain", "utf-8")
    msg["Subject"] = asunto
    msg["From"] = GMAIL_USER
    msg["To"] = destinatario_principal
    msg["Cc"] = ", ".join(cc)

    destinatarios_totales = [destinatario_principal] + cc

    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as servidor:
        servidor.login(GMAIL_USER, GMAIL_APP_PASS)
        servidor.sendmail(GMAIL_USER, destinatarios_totales, msg.as_string())

def enviar_correo_vencimientos(documentos):
    asunto = f"Alerta: {len(documentos)} permiso(s) por vencer o vencidos"
    cuerpo = _construir_cuerpo(documentos, "ALERTA DE VENCIMIENTO DE PERMISOS")
    _enviar(DESTINATARIO_PRINCIPAL, CC, asunto, cuerpo)
    print("Correo de vencimientos enviado correctamente.")

def enviar_correo_acuses(documentos):
    asunto = f"Acuses en seguimiento: {len(documentos)} documento(s)"
    cuerpo = _construir_cuerpo(documentos, "LISTADO DE ACUSES EN SEGUIMIENTO")
    _enviar(DESTINATARIO_ACUSES, CC_ACUSES, asunto, cuerpo)
    print("Correo de acuses enviado correctamente.")

if __name__ == "__main__":
    if not GMAIL_APP_PASS:
        raise RuntimeError(
            "No se encontro GMAIL_APP_PASS en el entorno. "
            "Configura la variable en el Programador de tareas."
        )

    vencimientos, acuses = escanear()

    if vencimientos:
        enviar_correo_vencimientos(vencimientos)
    else:
        print("No hay permisos proximos a vencer.")

    if acuses:

```

```
    enviar_correo_acuses(acuses)
else:
    print("No hay acusos proximos a vencer.")
```

4.5 carpetas.txt: administrar la lista blanca sin editar el script

Con 73 rutas y crecimiento activo (32 solo en esta actualización), cada carpeta nueva exigía antes editar `script.py`, validar la sintaxis y volver a colocar el archivo en el NAS — un riesgo innecesario para un cambio que es puramente de datos, no de lógica. Esto ya se resolvió: `CARPETAS` se externalizó a un archivo de texto plano, `carpetas.txt`, que vive junto a `script.py` y se lee al arrancar. Agregar o quitar una carpeta es ahora **editar una línea de texto** —desde File Station o cualquier editor—, sin tocar Python ni arriesgar un error de sintaxis que tumbé la tarea programada.

```
# carpetas.txt (extracto)
1.Permisos de Recolección y Traslado/2.Recicley MX S.A. de .C.V/Estado de México
1.Permisos de Recolección y Traslado/2.Recicley MX S.A. de .C.V/Queretaro
18.Cedula de Zonificación/1.Xhala 2
# las líneas vacías y las que inician con # se ignoran
```

```
def _cargar_carpetas(nombre_archivo=_ARCHIVO_CARPETAS):
    ruta = os.path.join(os.path.dirname(os.path.abspath(__file__)), nombre_archivo)
    with open(ruta, encoding="utf-8") as f:
        return [
            f"({_BASE_CALIDAD})/{linea.strip()}"
            for linea in f
            if linea.strip() and not linea.strip().startswith("#")
        ]

CARPETAS = _cargar_carpetas()
```

⚠ **Nota de despliegue:** `carpetas.txt` debe copiarse al NAS en la *misma carpeta* que `script.py` (hoy, `/volume1/scripts/`), porque la ruta se resuelve de forma relativa al script (`os.path.dirname(os.path.abspath(__file__))`). Si el archivo falta en esa carpeta, el script lanzará `FileNotFoundError` al arrancar y la tarea programada fallará por completo — conviene incluir `carpetas.txt` en el mismo paso de despliegue que `script.py`.

Alternativa descartada: generar `CARPETAS` automáticamente con `os.listdir` sobre cada carpeta base, en lugar de listar rutas explícitas. Se prefirió la lista blanca en archivo porque conserva el control explícito (evita monitorear subcarpetas nuevas no destinadas a documentos con fecha de vencimiento) y sigue siendo trivial de editar.

5. Buenas Prácticas de Seguridad Aplicadas

- La contraseña de la cuenta de correo nunca se escribe dentro del script de Python; se inyecta como variable de entorno desde un script intermediario al momento de la ejecución.
- Se utiliza una contraseña de aplicación de Google, específica para esta integración, en lugar de la contraseña principal de la cuenta.
- La conexión hacia el servidor de correo se realiza siempre bajo cifrado SSL/TLS.
- El acceso a la carpeta de scripts en el NAS está limitado al personal del departamento de TI.
- El script solo opera dentro de una lista blanca de rutas explícitas (`CARPETAS`), evitando el recorrido no controlado del resto del NAS.

6. Registro de Cambios

Actualización — 10 de agosto de 2026 (externalización de CARPETAS)

NUEVO La lista blanca CARPETAS se sacó del código: ahora se carga en tiempo de ejecución desde `carpetas.txt` (nuevo archivo, misma carpeta que `script.py`) mediante la función `_cargar_carpetas()`. Agregar o quitar una carpeta ya no requiere editar ni volver a desplegar el script (ver 4.5).

CAMBIO Se eliminaron de `script.py` las variables base intermedias que solo se usaban para construir CARPETAS (`_BASE_PERMISOS`, `_RECICLEY_MX`, `_RMO`, `_BASE_SEGREGACION`, `_SEGREGACION_RECICLEY_MX`, `_BASE_LICENCIAS`, `_LICENCIAS_FORANEOS`, `_LICENCIAS_TOLUCA`, `_BASE_APROVECHAMIENTO`, `_APROVECHAMIENTO_RMO`, `_BASE_ZONIFICACION`, `_BASE_FACTIBILIDAD`, `_BASE_NISS`, `_BASE_AGUA_DRENAJE`); solo queda `_BASE_CALIDAD`.

RIESGO OPERATIVO `carpetas.txt` debe copiarse al NAS junto con `script.py` en cada despliegue; si falta, el script no arranca (ver la nota en 4.5).

Actualización — 10 de agosto de 2026 (ampliación de la lista de carpetas)

NUEVO Se agregaron **32 rutas nuevas** a CARPETAS, correspondientes a 6 categorías nuevas del árbol de Calidad-Gestoría — 18. Cedula de Zonificación (18), 19. Movilidad Vial GDL y SLP (1), 21. NISS — Recibo Pago de agua y Saneamiento (7), 22. Poliza vs Daños Vigente al 30052027 (1), 23. Factibilidad de agua y drenaje (3), 24. Autorización para depósito de RSU en sitio SLP (1) — más una subcarpeta adicional (Guadalajara) en la categoría 20. Factibilidad Vial. La lista blanca pasa de 41 a **73 rutas** (ver 3.2).

Actualización — 10 de agosto de 2026 (sincronización inicial con `script.py`)

CAMBIO La lista blanca CARPETAS pasó de dos rutas de ejemplo genéricas a las **41 rutas reales de producción** entonces vigentes, agrupadas en 5 categorías dentro de `/volume4/Contraloría/1.Calidad-Gestoría`.

NUEVO Manejo independiente de **acuses de recibo**: documentos con prefijo `ACUSE_` se separan automáticamente y se reportan en un segundo correo (`enviar_correo_acuses`), con destinatarios propios

(DESTINATARIO_ACUSES, CC_ACUSES).

NUEVO El cuerpo del correo ahora se agrupa por carpeta y se ordena por fecha de vencimiento (`_agrupar_por_carpeta`), en lugar del listado plano anterior.

CAMBIO Destinatarios actualizados: DESTINATARIO_PRINCIPAL es ahora `legal.permisos@recicleymx.com` y CC es `leonardogr118@gmail.com`, en lugar de los correos de contraloría y calidad documentados previamente.

RIESGO El patrón de nombre de archivo (PATRON) ya **no tolera doble extensión** `.pdf.pdf`; ver el aviso en la sección 3.3.

CAMBIO Se homologó el nombre del script a `script.py` en todo el documento, conforme al archivo real observado en File Station (antes referido como `verificar_documentos.py`).

7. Próximos Pasos

1. Copiar `carpetas.txt` al NAS, junto a `script.py` en `/volume1/scripts/`, antes de la próxima ejecución programada — sin ese archivo el script no arranca.
2. Confirmar la periodicidad definitiva de ejecución (diaria o semanal) según la operación del área correspondiente.
3. Finalizar la configuración de Cloud Sync para mantener un respaldo automático de los documentos en Google Drive.
4. Resolver el caso de nombres con doble extensión `.pdf.pdf` detectado en la carpeta de pruebas (renombrar archivos o ajustar `PATRON`).
5. Evaluar, a futuro, la extracción de la fecha de vencimiento directamente del contenido del PDF, como capa adicional de validación.